SmartStock: Features

Oct 13, 2026 · @kai

The one-line pitch

SmartStock is a web app that gives small businesses one place to manage products, record sales, track stock automatically, and see how the business is performing, replacing scattered spreadsheets.

Features: technical and business

Each feature below pairs what it does for the business with how it is built.

1. Product and supplier management

- **Business value:** A single, clean catalog of what the business sells and who it buys from. This removes duplicate entries, inconsistent pricing, and "which spreadsheet is the latest?" confusion.
- **Technical:** CRUD operations on `inventory_products` with fields like SKU, price, cost, stock quantity, and reorder level. Suppliers link to products through a foreign key. Validation (unique SKUs, non-negative prices) is enforced at the database level with constraints, not just in the UI.

2. Sales recording with automatic inventory updates

- **Business value:** Staff record a sale once, and stock levels are correct immediately. This prevents overselling and the "we thought we had 20 left" problem.
- **Technical:** A sale is a parent-child structure: one row in `inventory_sales` (customer, total, timestamp) and many rows in `inventory_sale_items` (product, quantity, unit price at time of sale). Storing the price on the line item preserves historical accuracy when prices change later.
- **Atomicity:** The stock decrement and sale insert should run in a single database transaction (a Postgres function/RPC), so you never get a sale without a stock update, or the reverse.
- **Concurrency:** Use row locking or an atomic `UPDATE ... SET stock = stock - qty WHERE stock >= qty` to prevent race conditions when two cashiers sell the last unit at once.

3. Inventory tracking and stock movement history

- **Business value:** An audit trail answers "where did these 12 units go?" It helps detect shrinkage, theft, or data-entry errors, and it is essential for trust in the numbers.
- **Technical:** An append-only movement ledger (type: sale, restock, adjustment, return; quantity delta; reference ID; user; timestamp). Current stock can be derived from or reconciled against this ledger. Database triggers or the sale RPC write to it automatically.

4. Customer management

- **Business value:** Knowing who buys what enables repeat-customer analysis and, later, loyalty or credit features.

- **Technical:** `inventory_customers` is linked to sales by foreign key, with optional walk-in sales (nullable customer ID). Supports queries like lifetime value and purchase frequency.

5. Dashboard analytics with date filters

- **Business value:** Owners get answers without building reports: revenue today/this week/this month, top sellers, slow movers, low-stock alerts, and average order value.
- **Technical:** Aggregations (`SUM`, `COUNT`, `GROUP BY`) run in SQL, using views or RPC functions rather than pulling raw rows to the browser. Date-range filters are parameterized, and indexes on `sales.created_at` and `sale_items.product_id` keep queries fast as data grows. Charts are rendered client-side from the aggregated results.

KPI	Business question it answers
Revenue and gross profit	Are we making money?
Top-selling products	What should I never run out of?
Low-stock alerts	What do I need to reorder now?
Slow-moving stock	What's tying up my cash?
Average order value	Are customers buying more per visit?

6. Authentication and authorization (multi-user, roles)

- **Business value:** An owner can see everything, while a cashier can only record sales. This protects sensitive data like costs and profit and limits the damage of mistakes.
- **Technical:** Supabase Auth handles sign-up, login, and sessions (JWT). A profiles table maps each user to a role (e.g., owner, manager, staff). Row Level Security (RLS) policies enforce permissions in the database itself, so even a tampered client can't bypass them. The frontend adds route guards and hides UI by role, but RLS is the real security boundary.

Architecture

The stack is a React or Next.js frontend on top of Supabase, with the important rules living in the database.

- **Frontend:** React/Next.js (or your stack) with a charting library.
- **Backend:** Supabase (Postgres, Auth, RLS, RPC functions).
- **Key design decisions to highlight:** transactional sales, price snapshots on line items, ledger-based stock history, and database-level security.

A note on your current scope

Your four listed tables cover products, customers, sales, and sale items, but your scope also mentions suppliers, stock movement history, and roles. Those need additional tables, for example `inventory_suppliers`, `inventory_stock_movements`, and a user profiles/roles table. Either add them

to the build or trim the scope in your write-up, because reviewers will notice a mismatch between claimed and actual features.

Portfolio tips

- Lead with the problem and outcome, then the tech.
- Include screenshots of the dashboard and a short demo GIF of recording a sale and watching stock drop.
- Add a "challenges solved" section, such as concurrency on stock updates, RLS, and aggregation performance. This is what separates a strong project from a tutorial clone.
- Seed realistic demo data so the dashboard looks alive.